

浙江大学

实验报告

Final project

小组名：

项目名：打乒乓 PRIME

报告时间：2026/6/18

组长：王传宇

组员：齐思航

指导老师：蔡铭

Contents

实验报告	1
1 设计说明	5
1.a 功能概述	5
1.a.a 基础功能	5
1.a.b 额外功能 1——难度选择	5
1.a.c 额外功能 2——道具	5
1.a.d 额外功能 3——随机化发球角度	6
1.a.e 额外功能 4——蜂鸣器与拖尾	6
1.a.f 额外功能 5——PS/2 接口	6
1.a.g 额外功能 6——AI 对手	6
1.b 游戏逻辑	6
1.b.a 状态机设计	6
1.b.b 球的运动与碰撞逻辑	7
1.b.c 反弹角度计算	7
1.b.d 技能逻辑	8
1.c 外设使用	8
1.c.a 七段数码管	8
1.c.b 蜂鸣器	9
1.c.c LED	9
1.c.d PS/2 键盘接口	9
1.c.e VGA	11
2 核心模块说明	11
2.a defines.vh——全局宏定义头文件	11
2.b Top.v——顶层模块	12

2.b.a	主要功能	12
2.b.b	信号流	12
2.c	game_logic.v——游戏逻辑状态机	13
2.c.a	game_tick 分频控制	13
2.c.b	宽挡板道具	13
2.c.c	边沿检测	13
2.d	vga_render.v——VGA 图像渲染	13
2.d.a	球的拖尾效果	14
2.d.b	分段数字显示	14
2.d.c	GAME OVER 文字	14
2.d.d	渲染优先级	15
2.e	vgac.v——VGA 时序控制器	15
2.e.a	计数器	15
2.e.b	同步信号	15
2.e.c	坐标输出	16
2.e.d	颜色输出	16
2.f	ai_paddle.v——AI 对手	16
2.f.a	追踪与死区	16
2.f.b	随机延迟	16
2.f.c	空闲震荡	17
2.f.d	回中行为	17
2.g	其余模块	17
3	仿真、调试过程分析	17
3.a	ai_paddle	17
3.b	game logic	18

3.c	buzzer_ctrl	19
3.d	vga_render	19
3.e	ps2_keyboard	19
3.f	Top	20
4	小组主要工作说明	21
4.a	参考工程	21
4.b	设计方面	21
4.c	调试方面	21
5	各成员贡献比例	22

1 设计说明

1.a 功能概述

本项目旨在实现打乒乓小游戏，并在其基础上进行优化并添加额外内容，增加其可玩性的同时提高游玩的舒适度。

1.a.a 基础功能

该游戏为双人对战小游戏（也可以单人与 AI 对战），每人操作一个球拍进行接球，用 w 和 s 控制其上下移动。球遇到上下边界与球拍会反弹，没接到球者判负。实时记录双方得分，得分先到达 11 分者获胜，同时屏幕上显示 GAME OVER。

1.a.b 额外功能 1——难度选择

可利用 SW[2:3] 选择游戏难度。游戏分为 4 个难度：Easy、Hard、Master、Auto。其中 Easy、Hard、Master 中球速逐渐提升，而 Auto 比较特殊，初始速度和 Easy 一致，但每次击球会使球速变为原来的 1.1 倍，更加紧张刺激（）

同时，选择的难度会在四个七段数码管上显示

1.a.c 额外功能 2——道具

比赛每隔一段时间会在双方球拍上下位置生成神秘小道具，吃（碰）到道具者的球拍会短时间延长（延长的部分为灰色），使接球更加容易。

1.a.d 额外功能 3——随机化发球角度

每次发球的角度都会在一定范围内随机化，让双方选手开局就进入状态。

1.a.e 额外功能 4——蜂鸣器与拖尾

击球、失分、结束都会有不同的蜂鸣器音效，同时球会有拖尾的视觉效果来体现其速度感。

1.a.f 额外功能 5——PS/2 接口

本游戏可在键盘上进行游玩，改善了双人对战的游戏体验 提供一套较为简单的 AI 行为逻辑，实现对右挡板的控制

1.a.g 额外功能 6——AI 对手

1.b 游戏逻辑

1.b.a 状态机设计

状态机设计如下：

状态	编码	功能
S_IDLE	3'd0	空闲：等待开始，球和挡板居中
S_SERVE	3'd1	发球：球居中，随机选择发射角度；等待按键或超时自动发球
S_PLAY	3'd2	游戏进行：球移动、碰撞检测、挡板控制
S_PAUSE	3'd3	暂停：一切冻结，等待继续
S_SCORE	3'd4	得分暂停：约 1 秒后回到发球
S_OVER	3'd5	游戏结束：显示 GAME OVER，等待按键重置

状态转移：

- IDLE → (按开始) → SERVE → (按开始/超时) → PLAY → (按开始) → PAUSE → (按开始) → PLAY
- PLAY → (球出界) → SCORE → (超时) → SERVE
- PLAY → (球出界+满 11 分) → OVER → (按开始) → IDLE
- 任意状态 → (Esc 软复位) → IDLE

1.b.b 球的运动与碰撞逻辑

- 速度表示: ball_dx (水平速度) 和 ball_dy (垂直速度) 均为 11 位有符号数, 支持负方向
- 位置更新: $\text{next_x_s} = \text{ball_x} + \text{ball_dx}$, $\text{next_y_s} = \text{ball_y} + \text{ball_dy}$, 使用有符号运算避免溢出
- 碰撞逻辑:
 1. 上下边界反弹: 当 $\text{next_y_s} \leq 0$ 或 $\text{next_y_s} \geq 464$ 时, ball_dy 取反
 2. 左挡板碰撞: 当球右缘进入左挡板区域 ($\text{next_x_s} \leq 30 \ \&\& \ \text{next_x_s} + 8 \geq 20$) 且 Y 坐标与挡板重叠时触发
 3. 右挡板碰撞: 类似, 检测球左缘与右挡板 ($\text{next_x_s} + 8 \geq 610 \ \&\& \ \text{next_x_s} \leq 620$) 的重叠
 4. 出界得分: 若球未碰挡板且 $\text{next_x_s} \leq 0$ (右方得分) 或 $\text{next_x_s} + 8 \geq 640$ (左方得分)

1.b.c 反弹角度计算

碰撞时根据球心与挡板中心的偏移量确定反弹角度:

偏移量	角度索引	效果
> 20px	±2	大角度: dx=4, dy=±2
> 5px	±1	中角度: dx=5, dy=±1
-5 - +5px	0	水平: dx=5, dy=0
< -5px	-1	中角度反向
< -20px	-2	大角度反向

1.b.d 技能逻辑

两个状态：

- S_COOLDOWN (冷却)：等待一段时间后生成新道具
- S_ACTIVE (激活)：道具可见，检测碰撞或超时

冷却时间：

- 基础冷却：COOLDOWN_BASE = 480 个 game_tick $\approx$ 8 秒
- 随机附加：rand_cnt[5:0] (0 63)，总冷却 $\approx$ 8 9 秒

道具生成：

- X 位置：交替出现在左/右挡板区域的中线处
 - 左：LEFT_PADDLE_X + PADDLE_W/2 = 25
 - 右：RIGHT_PADDLE_X + PADDLE_W/2 = 615
- Y 位置：40 + rand_cnt[9:4] $\times$ 4，范围 40 292，随机化

碰撞检测：

道具为 6 $\times$ 6 像素，检测其与挡板的 Y 范围重叠：

```
1 (powerup_y + 6) > paddle_y && powerup_y < (paddle_y + PADDLE_H)
```

碰撞后产生 hit_left 或 hit_right 脉冲，通知 game_logic 激活宽挡板。

存在时间：

LIFETIME = 150 个 game_tick $\approx$ 2.5 秒，超时后消失进入冷却。

1.c 外设使用

1.c.a 七段数码管

用于显示当前游戏难度，分为 Easy、Hard、Master、Auto 四种

1.c.b 蜂鸣器

驱动无源蜂鸣器产生三种不同频率的方波音效。

事件	频率	半周期计数	持续时间
击球 hit_paddle	1000Hz	12587	0.2 秒
得分 score_event	1500Hz	8391	0.2 秒
结束 game_over	2000Hz	6293	0.2 秒

方波生成原理

- half_period_cnt: 从 0 计数到 current_half - 1
- 计满时 note_phase 翻转 → 产生方波
- 方波频率 = $25.175\text{MHz} / (2 \times \text{current_half})$

1.c.c LED

8 个 LED 指示灯映射如下：

LED 位	含义	表现
0	空闲 IDLE	常亮
1	发球 SERVE	常亮
2	游戏中 PLAY	常亮
3	发球方	0=左, 1=右
4	击球	快闪
5	得分	快闪
6	暂停 PAUSE	慢闪 约 0.5Hz
7	游戏结束 OVER	常亮

闪烁效果通过 blink_cnt（24 位计数器，计 12.5M 次 ≈ 0.5 秒）和 blink_phase 实现。

1.c.d PS/2 键盘接口

PS/2 协议是 11 位异步串行协议：

- 1 位起始位 (0)

- 8 位数据位 (LSB first)
- 1 位奇校验位
- 1 位停止位 (1)

数据在 PS/2 时钟的下降沿采样。

- 帧接收

bit_cnt 从 0 计数到 10:

- bit_cnt=0: 跳过起始位
- bit_cnt=1~8: 将数据位移入 shift_reg
- bit_cnt=9: 跳过校验位
- bit_cnt=10: 帧完成, 产生 frame_done 脉冲

扫描码解码

PS/2 键盘使用“Make/Break”机制:

- 按键按下: 发送 Make 码 (如 W=0x1D)
- 按键释放: 发送 0xF0 + Make 码 (Break 序列)
- 扩展键 (方向键等): 前缀 0xE0

状态变量:

- is_extended: 收到 0xE0 前缀标志
- is_break: 收到 0xF0 前缀标志
- key_valid: 解码完成脉冲

键位映射

按键	扫描码	功能
W	0x1D	左挡板上移
S	0x1B	左挡板下移

按键	扫描码	功能
↑ (E0,75)	E0 75	右挡板上移
↓ (E0,72)	E0 72	右挡板下移
Space	0x29	开始/暂停
Enter	0x5A	开始/暂停
Esc	0x76	软复位

当收到 Make 码时对应信号置 1，收到 Break 码时置 0。

1.c.e VGA

用于渲染画面，时序控制器为 vgac.v

2 核心模块说明

2.a defines.vh——全局宏定义头文件

用于定义全项目通用的参数，方便统一管理，主要宏定义如下：

类别	宏名示例	作用
显示几何	SCREEN_W/H	VGA 分辨率 640×480
挡板参数	PADDLE_W/H, LEFT/ RIGHT_PADDLE_X	挡板宽 10px、高 80px、左右各距边界 20px
球参数	BALL_SIZE	球为 8×8 正方形
游戏规则	MAX_SCORE	先得 11 分者获胜
AI 参数	AI_DEAD_ZONE, AI_UPDATE_BASE/RANGE	死区 60px、更新间隔 24+随机 0 15 帧
难度等级	TICK_THRESH_SPEED1~5	控制 game_tick 频率:60Hz / 120Hz / 180Hz / 240Hz / 300Hz

类别	宏名示例	作用
计时参数	TICK_MAX, SCORE_TIMEOUT, SERVE_TIMEOUT	game_tick 周期约 419583 时钟 $\approx 60\text{Hz}$; 得分后暂停 60 帧 $\approx 1\text{s}$; 自动发球超时 60 帧
数码管扫描	SCAN_MAX	25.175MHz / 6294 $\approx 4\text{kHz}$ 扫描频率
文字缩放	TEXT_SCALE	GAME OVER 字体放大系数, $2 = 16 \times 32\text{px}/\text{字符}$

2.b Top.v——顶层模块

2.b.a 主要功能

Top.v 主要负责的功能如下：Top.v 是整个系统的最高层，负责：

- 时钟生成：调用 clk_wiz 将 100MHz 主时钟转为 25.175MHz VGA 像素时钟
- 复位管理：双级同步器 rst_s1 $\rightarrow$ rst_s2 $\rightarrow$ rst_n，确保 PLL 锁定和 SW[0] 开关复位信号同步释放
- 模块例化与连接：像“主板”一样把所有子模块连在一起

2.b.b 信号流

- 输入路径：矩阵键盘/PS/2 键盘 $\rightarrow$ input_merger (OR 逻辑合并) $\rightarrow$ game_logic
- 游戏路径：game_logic 输出球坐标、挡板坐标、分数、状态 $\rightarrow$ vga_render 生成像素 $\rightarrow$ vgac 输出时序和颜色
- 外设路径：game_logic 的事件脉冲 $\rightarrow$ buzzer_ctrl (声音) / led_status (LED 指示) / seg_display (难度显示)
- 开关信号：SW[1]=AI 使能，SW[3:2]=难度选择

2.c game_logic.v——游戏逻辑状态机

该模块功能主要分为以下几部分：状态机的设计（前文已述）、game_tick 分频控制、球运动与碰撞（已述）、反弹角度计算（已述）、挡板道具、边沿检测几部分

2.c.a game_tick 分频控制

使用 19 位计数器 tick_counter，从 0 计数到 tick_threshold。tick_threshold 根据难度设定不同值：

- Easy: 419583 → 约 60Hz
- Hard: 209791 → 约 120Hz
- Master: 139861 → 约 180Hz
- Auto: 初始 60Hz，每次击球乘以 $\times(10/11)$ ，即加速约 1.1 倍

2.c.b 宽挡板道具

当 pw_hit_left 或 pw_hit_right 脉冲到来时，wide_timer 设为 300 个 game_tick（约 5 秒），期间挡板上下各扩展 5 像素（灰色显示）。

2.c.c 边沿检测

start_pause 和 soft_reset 信号需要边沿检测（start_pause_d 延迟一拍后比较），确保一次按键只触发一次状态转换，而不是连续触发。

2.d vga_render.v——VGA 图像渲染

vga_render 是“画师”模块，根据当前扫描到的像素坐标和各种游戏对象坐标，逐像素计算颜色输出。

2.d.a 球的拖尾效果

使用三级移位寄存器保存球的前三帧位置：

```
1 game_tick 有效时：  
2   bx1 <= ball_x (当前帧) → bx2 <= bx1 (前一帧) → bx3 <= bx2  
   (前两帧)
```

渲染时按优先级叠加：

- bx3/by3：最暗拖尾（12'h333，深灰）
- bx2/by2：中等拖尾（12'h777，中灰）
- bx1/by1：较亮拖尾（12'hBBB，浅灰）
- ball_x/ball_y：当前球（12'hFFF，白色，覆盖拖尾）

2.d.b 分段数字显示

每个数字由 16 行 × 8 列的位图（bitmap）表示，存储在 digit_bitmap 寄存器数组中。0 9 的位图是硬编码的点阵字体 ROM。

显示位置：

- 左方分数十位：(200, 30)，个位：(216, 30)
- 右方分数十位：(408, 30)，个位：(424, 30)

渲染逻辑：判断当前像素是否落在某个数字区域内 → 计算行内偏移和列内偏移 → 从位图中取出对应位 → 决定是白还是黑。

2.d.c GAME OVER 文字

“GAME OVER”由 9 个字符（含空格）组成，每个字符也是 8×16 点阵字体。通过 TEXT_SCALE 宏（=2）将每个字符放大到 16×32 像素。

居中计算：

- $\text{GAMEOVER_W} = 9 \times 16 = 144\text{px}$
- $\text{GAMEOVER_X} = (640 - 144) / 2 = 248$
- $\text{GAMEOVER_Y} = (480 - 32) / 2 = 224$

仅在 `game_state == S_OVER` 时显示，优先级最高。

2.d.d 渲染优先级

从低到高叠加（后写的覆盖先写的）：

1. 黑色背景
2. 中线虚线（`col 318~320, row[3:0] < 8`）
3. 球拖尾（由暗到亮）
4. 当前球（白色）
5. 挡板（白色主体 + 灰色扩展部分）
6. 道具菱形（绿色，去角）
7. 分数数字（白色）
8. GAME OVER 文字（白色，最高优先级）

2.e vgac.v——VGA 时序控制器

`vgac.v` 是 VGA 信号的“行场同步发生器”，产生标准的 $640 \times 480 @ 60\text{Hz}$ VGA 时序。

2.e.a 计数器

- `h_count`：水平计数器，0 799（共 800 个时钟周期，含消隐区）
- `v_count`：垂直计数器，0 524（共 525 行，含消隐区）

`h_count` 每 25.175MHz 时钟加 1，到 799 归零；每当 `h_count` 归零时 `v_count` 加 1。

2.e.b 同步信号

- 行同步 `hs`：`h_count > 95` 时为高（同步脉宽 96 像素）
- 场同步 `vs`：`v_count > 1` 时为高（同步脉宽 2 行）

- 显示使能 rdn (低有效): 当 $143 < h_count < 783$ 且 $34 < v_count < 515$ 时有效 (即 640×480 可见区域)

2.e.c 坐标输出

- $row_addr = v_count - 35$ (减去消隐区偏移, 得到 0 479)
- $col_addr = h_count - 144$ (减去消隐区偏移, 得到 0 639)

2.e.d 颜色输出

12 位输入 $d_in = \{B[3:0], G[3:0], R[3:0]\}$, 仅在 $rdn=0$ (可见区域) 时输出到 RGB 引脚, 消隐区输出 0。

2.f ai_paddle.v——AI 对手

AI 有如下行为:

2.f.a 追踪与死区

- 计算球的中心 Y 和挡板中心 Y
- 死区 $AI_DEAD_ZONE = 60px$: 当球中心在挡板中心 $\pm 60px$ 范围内时, AI 不主动追踪
- 死区外: 球在上方则 $move_up$, 球在下方则 $move_down$

2.f.b 随机延迟

- 自由运行计数器 $rand_cnt$ 不断递增
- $update_timer$ 从 $(AI_UPDATE_BASE + rand_cnt[3:0])$ 倒数到 0
- 仅当 $update_timer == 0$ 时 AI 才重新采样球位置并决策
- 这使 AI 表现“犹豫”, 降低难度

2.f.c 空闲震荡

当球远离或球在死区内时，AI 进行小幅上下振荡（osc_phase 每 32 帧翻转方向），模拟“活”的表现而不会呆立不动。

2.f.d 回中行为

当球远离 AI 时，挡板缓慢回到屏幕中心（Y=240），为下次防守做准备。

2.g 其余模块

- clk_wiz.v: 用于时钟管理，输出 VGA 所需的像素时钟
- input_merger.v: 使用 OR 运算将矩阵键盘和 PS/2 键盘的信号合并
- keypad_scanner.v: 实现矩阵键盘的逐行扫描和去抖动
- led_status.v: LED 状态指示
- seg_display.v: 七段数码管显示，原理不再赘述
- powerup_ctrl.v: 道具控制器，前文已述

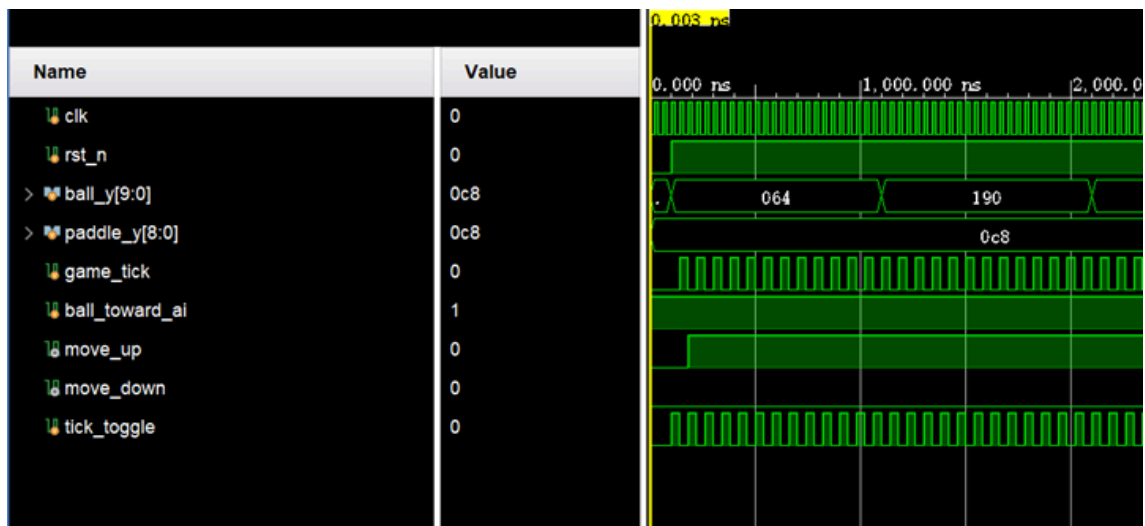
3 仿真、调试过程分析

3.a ai_paddle

该模块仿真分四部分：

- ball_y=100（球在上方），paddle_y=200
- ball_y=400（球在下方），paddle_y=200
- ball_y=236（死区内），paddle_y=200
- ball_y=231（略高于死区），paddle_y=200

测试其上方追踪、下方追踪、死区静止、随即延迟功能，部分仿真波形图如下：



仿真结果符合预期

3.b game logic

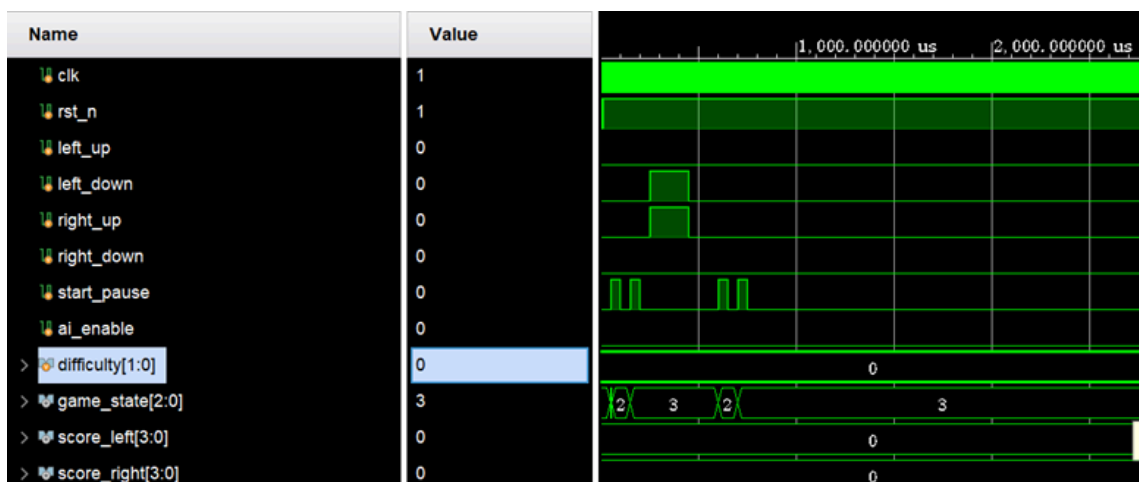
ai_paddle 之上的模块，负责游戏综合逻辑

仿真思路为模拟一局游戏

临时修改 TICK_MAX 与 SCORE_TIMEOUT，加快 game_tick 触发，进而加快仿真进程

遵循游戏进程：复位与空闲->开始游戏->开球->玩家控制->得分与重置->暂停与恢复

通过观察 game_state 验证游戏进程是否正常进行



可见游戏进程正常进行，从空闲到开始到暂停

3.c buzzer_ctrl

激活三类事件（击球、得分、结束）的脉冲，检查 buzz 输出

3.d vga_render

验证 vga_render 模块能否在正确的位置显示出正确的颜色

测试阶段	操作	验证目标
初始化	设置球在 (320,240)，球拍在 200，分数 3:7，状态为 PLAY	建立已知初始状态
Test 1	读取背景位置 (100,100)	验证背景色是否为黑色 (12'h000)
Test 2	读取球的位置 (320,240)	验证球是否为白色 (12'hFFF)
Test 3	读取分数位置 (200,30)	观察分数数字是否正确显示
Test 4	切换到 GAME OVER 状态,读取 'G' 的左上角 (284,232)	验证 GAME OVER 文字是否为白色
Test 5	切换回 PLAY 状态	验证 GAME OVER 文字是否消失

3.e ps2_keyboard

- 模拟 PS/2 总线时序（时钟 16.7kHz，数据帧格式）
- 发送按键的 Make Code（按下）和 Break Code（释放）
- 验证解码后的按键信号（left_up、left_down、right_up、right_down、start_pause）

步骤	发送的扫描码	对应按键	预期输出
1	0x1D (make)	W 键按下	left_up = 1
2	0xF0 + 0x1D (break)	W 键释放	left_up = 0
3	0x1B (make)	S 键按下	left_down = 1
4	0xF0 + 0x1B (break)	S 键释放	left_down = 0
5	0xE0 + 0x75 (make)	上箭头 按下	right_up = 1
6	0xE0 + 0xF0 + 0x75 (break)	上箭头 释放	right_up = 0
7	0x5A (make)	Enter 按下	start_pause = 1
8	0xF0 + 0x5A (break)	Enter 释放	start_pause = 0

3.f Top

顶层模块测试

阶段	测试内容	验证点	方法
Phase 0	上电复位	系统正确复位	拉低 rst_sw, 等待 MMCM 锁定
Test 1	IDLE 状态	复位后进入空闲状态	检查 game_state=0, LED[0]=1
Test 2	IDLE→SERVE	按开始键进入发球状态	检查 game_state=1, LED[1]=1
Test 3	SERVE→PLAY	再次按开始进入游戏状态	检查 game_state=2, LED[2]=1
Test 4	球运动	球是否开始移动	等待 10ms, 检查 ball_x/y 变化
Test 5	球拍控制	矩阵键盘控制球拍	按下左球拍下移, 右球拍上移
Test 6	暂停/恢复	暂停功能	按开始键进入 PAUSE, 再按恢复
Test 7	边界反弹	球碰到上下边界反弹	检查 ball_y 不超出屏幕边界

Test 8	得分事件	球出界触发得分	等待球出界，检查分数变化
Test 9	7 段数码管	显示是否正常	检查 AN 是否在扫描，SEGMENT 有值
Test 10	VGA 同步	VGA 信号是否产生	检查 vga_hs/vs 不是 X/Z
Test 11	蜂鸣器	蜂鸣器信号是否驱动	检查 buzzer 不是 X/Z
Test 12	AI 模式	AI 是否正确追踪球	启用 AI (SW[1]=1)，检查球拍运动

4 小组主要工作说明

4.a 参考工程

4.b 设计方面

- 设计难度分级、设置不同难度球速
- 设计道具（增加球拍宽度）
- 随机化发球角度，改进游戏发球机制
- 设计蜂鸣器音效
- 改善视觉效果，新增拖尾特效
- 利用 PS/2 接口接入键盘
- 设计 AI 对手，增加单人模式

4.c 调试方面

负责新老模块的调试与整合

5 各成员贡献比例

王传宇 贡献 60%

齐思航 贡献 40%

附图：